

AUDIT PROPOSAL 2

This proposal covers the scope of the audit, provides a quick orientation, and introduces the workflow used to build/test our code.

The code is available at <https://github.com/wolvesofwallstreet/wolves.finance>.

Scope

The audit will cover our ERC-1155 token contracts, and investment contracts for WOWS/ETH UniswapV2 (wolf investments) and Curve Y Pool (dog investments).

- WowsErc1155.sol *)
- WowsCryptofolio.sol *)
- WowsSftMinter.sol
- TradeFloor.sol
- RewardHandler.sol
- Sftevaluator.sol
- Controller.sol
- CFolioFarm.sol
- CFolioItemHandlerLP.sol
- CFolioItemHandlerSC.sol

*) Contracts are audited ([Link to Audit](#)) and comments from the audit are partially implemented. The contracts deployed on mainnet are audit git SHA state.

Orientation

WowsErc1155.sol is our ERC-1155 token. We don't make use of fungible tokens, so all tokens are NFTs (SFT). Our tokens act as cryptofolios (cfolio), and can contain other NFTs (CFolioItems).

SFT tokens minted here are meant to not be tradeable on NFT marketplaces because they have underlying value which could be changed while it is on auction. This is ensured by reverting Approval calls for unregistered contracts.

TokenId space is divided into 2 main sections, where the first section is divided into 2 sub-sections:

- stock base cards (< 0x100000000)
- custom base cards (>=0x100000000 and < 0x10000000000000000)
- CFolioItems (>=0x10000000000000000) and < 128 Bit, where "base cards" describe our root cards (wolves / bois).

WOWSCryptofolio.sol is our cfolio “template”. This contract is deployed once on mainnet, and when an ERC-1155 SFT token is minted, the contract is cloned. This address of the cloned contract is what receives CFolioItem NFTs being added to the cfolio.

WOWSSftMinter.sol is our SFT crowdsale. We’ve created a simple purchase dapp where users can buy our first round of SFTs. A user can mint an SFT and pay using our ERC-20 WOWS token.

This contract was extended after the first audit and provides now one click CFolioItem purchase.

TradeFloor.sol is our NFT contract which “locks” WOWSERC1155 minted SFTs to make them tradeable in our and third-party NFT marketplaces.

Technically speaking, “locking” is nothing more than transferring ownership of an ERC-1155 SFT to the TradeFloor contract and minting a new Tradefloor NFT with a new tokenId in return to the caller’s account. The tokenId is constructed using the SFT tokenId (128 Bit) and a hash generated from underlying value (128 Bit). The reason for using a hash part is to prevent frontrun vulnerabilities while a token is on sale at NFT marketplaces. After locking the underlying value of the SFT is not accessible by the user anymore.

An SFT can be unlocked by burning the TradeFloor NFT. Doing so, the TradeFloor NFT is burned, and in return the locked ERC-1155 SFT is transferred back to the user’s account. After this procedure the ERC-1155 SFT is “unlocked”.

TradeFloor.sol is deployed using the UpgradeableProxy concept. We have implemented code for compatibility with third-party NFT marketplaces. These conditions can change and we have to be able to upgrade the logic.

RewardHandler.sol is responsible for distributing funds according to our tokenomics.

Right now there are 3 situations when rewards have to be distributed:

1. User claims WOWS rewards from our currently-running WOWS/ETH LP staking contract (temporary contract not in the scope of this audit),
2. Purchase of WOWSERC1155 SFT tokens in WOWSSftMinter contract.
3. User claims rewards from their WOWSERC1155 SFT if it contains CFolioItems which are rewardable.

In earlier versions of the contract, the distribution was specified by the caller, but this has changed and the distribution is now fixed (distribute2()).

SFTEvaluator.sol calculates the % weight of a base card and notifies all reward-emitting CFolioItemHandler in case the % weight of a base card has changed.

Base cards are purchased with an initial weight (depending the card level), but this weight can be changed due to time-based triggers (holding the card longer than 2 months) or purchase of CFolioItems which increase the % weight.

Currently this % weight is used to calculate the rewards in our WOWS reward farms.

Controller.sol is responsible for controlling all WOWS-emitting reward farms.

Besides managing fees which could be charged on deposit / withdraw / claim, it also accepts meta transactions that refuel the farm rewards or (not currently implemented) rebalance farm investments on a regular basis.

CFolioItemHandlerLP.sol handles UniV2 WOWS/ETH LP tokens staked in a CFolioItem SFT.

The contract allows adding and removing of LP tokens into an unlocked CFolioItem SFT.

Beside this, the contract evaluates the amount of LP tokens which are in a locked NFT and are owned by a base SFT. This amount is used in the CFolioFarm to earn rewards.

LP tokens staked are owned by the CFolioItemHandlerLP contract. The overall amount of LP tokens a user has provided for a given CFolioItem SFT are stored in CFolioFarm (CFolioFarm::addAssets) using the CFolioItem SFT address.

The rewardable amounts calculated by iterating over all CFolioItems in a base SFT are also stored in CFolioFarm using the baseSFT address, but added / removed using CFolioFarm::addShares which adds them to the reward-emitting logic.

CFolioItemHandlerSC.sol handles stablecoin (SC) and Curve Y pool (yCrv) investment tokens deposited in a CFolioItem SFT.

The contract allows adding and removing of SC and yCrv tokens into an unlocked CFolioItem SFT.

SC tokens are converted to and from yCrv by calling an external Curve contract.

yCrv tokens staked are owned by the CFolioItemHandlerSC contract. The overall amount of yCrv tokens a user has provided for a given CFolioItem SFT are stored in CFolioFarm (CFolioFarm::addAssets) using the CFolioItem SFT address.

The rewardable amounts calculated by iterating over all CFolioItems in a base SFT are also stored in CFolioFarm using the baseSFT address but added/removed using CFolioFarm::addShares which adds them to the reward-emitting logic.

CFolioFarm.sol is a generic reward farm that emits WOWS tokens through Controller.sol.

The contract is based on the synthetics.io reward farm and needs regular refuelling/adjustment of rewards.

We added 2 functions (add-/removeAssets) which store state from the CFolioItemHandler, but are not counted toward the reward logic. We want to have the CFolioItemHandler contract be upgradeable, and store as little state as possible. More about this in the Workflow section.

Workflow (user view)

Purchase SFT

- User selects an NFT stock card in our dapp. Each stock card in the UI has a level and a cardId. A base tokenId is calculated by: $\text{tokenIdBase} = (\text{level} \ll 24) \mid (\text{cardId} \ll 16)$. In hexadecimal, this is 0xLLCC0000, where L is level and C is cardId. The lower 16 bits are used for quantities.
This means we can manage 256 levels, 256 cardIds per level and 65536 quantities per cardId.
- User calls the SFTMinter contract which: a.) requests the next unminted tokenId from the 65536 possible for a given level and cardId, and b.) mints this token. Both are function calls (getNextMintableTokenId() / mint()).

When a token is minted for the first time, a clone of WOWSCryptofolio is created and 1:1 referenced to this newly minted tokenId. This WOWSCryptofolio will later store information about CryptofolioItems owned by itself.

Purchase CFolioItem (UniV2 WETH/WOWS LP token stake)

- User provides liquidity into the Uniswap WETH/WOWS liquidity pool
- User selects one of eight LP staking investment cards (CFolioItem)
- User selects the base card (wolf only for LP staking) into which the CFolioItem should be inserted (see "Purchase SFT").
- User inserts the amount of LP tokens to stake into an input field.
- Pressing the "Buy CryptoFolioItem" will call `WOWSSftMinter::mintCFolioItemSFT()`, which executes following internal tasks:
 - Approve `WOWSSftMinter` in `WOWS` token contract
 - Approve `WETH/WOWS` LP tokens in `CFolioItemHandlerLP` contract
 - Transfer `WOWS` from user wallet to `WOWSSftMinter` and distribute
 - Mint `CFolioItem SFT`
 - Transfer `WETH/WOWS` LP tokens to `CFolioItemHandlerLP` contract
 - Lock the `CFolioItem SFT` into the `cfolio` of the base SFT
- User earns rewards because a locked `CFolioItem` is in the base SFT `cfolio`.

Purchase CFolioItem (stablecoin deposit)

- User selects one of eight SC investment cards
- User selects the base card (dog only for SC farming) into which the CFolioItem should be inserted.
- User inserts the amount of SC tokens or `yCrv` to deposit into an input field.
- Pressing the "Buy CryptoFolioItem" will call `WOWSSftMinter::mintCFolioItemSFT()`, which executes following internal tasks:
 - Approve `WOWSSftMinter` in `WOWS` token contract
 - Transfer `WOWS` from user wallet to `WOWSSftMinter` and distribute
 - Mint `CFolioItem SFT`
 - Transfer `SC` and `yCrv` tokens to `CFolioItemHandlerSC` contract
 - Lock the `CFolioItem SFT` into the `cfolio` of the base SFT
- User earns rewards because a locked `CFolioItem` is in the base SFT `cfolio`.

Trade base SFT on NFT marketplace

- User locks the purchased SFT in the `TradeFloor` contract (there is a button in the UI)
- In return the user gets an NFT from `TradeFloor` which can be traded in e.g. OpenSea

Trade CFolioItem on NFT marketplace

- User has to transfer the locked `CFolioItem` from base SFT `cfolio` into their wallet
- User can now head over to e.g. Opensea and trade their `CFolioItem` NFT

Add more Univ2 WETH/WOWS LP tokens into an existing CFolioItem

- User has to transfer the locked `CFolioItem` from the base SFT `cfolio` into their wallet.

- User has to unlock the locked CFolioItem (by burning the TradeFloor NFT) to get in return the SFT CFolioItem.
- User calls deposit() in the CFolioItemHandlerLP contract.
- After successfully adding LP tokens, the user has to lock and transfer the NFT back to the base SFT cfolio to earn rewards again.
- We'll implement helper functions which bundle some of the calls.

Add more SC or yCrv tokens into an existing CFolioItem

- User has to transfer the locked CFolioItem from the base SFT cfolio into their wallet.
- User has to unlock the locked CFolioItem (by burning the TradeFloor NFT) to get in return the SFT CFolioItem.
- User calls deposit() in the CFolioItemHandlerSC contract.
- After successfully adding SC or yCrv tokens, the user has to lock and transfer the NFT back to the base SFT cfolio to earn rewards again.
- We'll implement helper functions that bundle some of the calls.

Workflow (unit test view)

Our build system is based on Yarn, with some additional tooling for dependencies. See the main README for available build system steps.

Our yarn steps are standard, except for the command `yarn depends`. This step invokes a custom build system that downloads dependencies, patches code, builds Vyper contracts, and then copies the Solidity and bytecode files to our contracts directory.

The custom tooling expects standard Linux tools like bash and patch. These can be installed on Windows, but it might be easier to build in a Linux VM.

Our unit tests are executed via `yarn test-chain`.

Setup

1.) Prerequisites:

- `yarn depends` and `yarn compile`
- 4 accounts (deployer / marketingwallet / teamwallet / test) where the marketingwallet and teamwallet are gnosis 4/6 multisig wallets on rinkeby and mainnet.
marketingwallet is in general the administrator of our contracts. All maintenance operations on the contracts are only accessible through this account.

2.) Execute `yarn deploy`. Note: the deploy script reads

`src/config/addresses.json` file and only deploys contracts which are not set in there. To override this behaviour call `IGNORE_ADDRESSES=1 yarn deploy`
If this step has successfully ran, marketingwallet should have 3600 WOWS tokens, and teamwallet should have 7500 WOWS tokens.

All contract addresses deployed can be found in `src/config/generated_addresses.json`.

3.) After deployment, the newly deployed Crowdsale contract opens after 5 minutes, and closes after another 5 minutes. To get some UniV2 WETH/WOWS LP tokens, and to unlock the UniV2 liquidity pool, perform following steps:

- call `Crowdsale.sol::setClosingTime()` with a timestamp in the future
- transfer 3 ETH to the Crowdsale contract from any account
- call `Crowdsale.sol::setClosingTime()` with a timestamp in the past
- call `Crowdsale.sol::finalizePresale()`

The last call will enable the Uniswap Liquidity pool, and also provides 3 ETH + WOWS as initial liquidity. The UniV2 LP tokens are transferred to the marketingwallet and can be used from there for testing.

An example is in our test suite.

4.) Mint some test SC tokens if deploying on a testnet. Minting is permissionless for quick testing. An example is in the deployment scripts. Minting can easily be done on Etherscan for quick refilling during testing.

5.) (If running test suite) Seed the Y pool by depositing SC tokens. An empty pool will change the amounts of SC tokens that the test runner can withdraw. An example is in the deployment scripts.

Additionally to 2.) For audit testing on Kovan we suggest setting only the Uniswap addresses in `src/config/addresses.json`:

```
"weth": "0xd0A1E359811322d97991E03f863a0C30C2cF029C",  
"uniV2Factory": "0x5C69bEe701ef814a2B6a3EDD4B1652CB9cc5aA6f",  
"uniV2Router": "0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D",
```

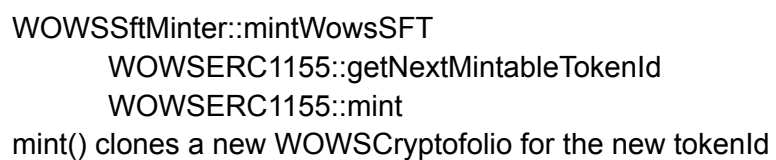
Having these 3 entries in the file will prevent deploying a new uniswap system.

Instead you can use the existing one and also the UniswapV2 GUI.

The Crowdsale calls described in 3.) can be found in the `test/web3/` folder.

The example in our test chooses a different initial investment: Provide 6.75 ETH and directly get WETH/WOWS LP tokens in the calling account wallet, instead of the 3 ETH method described above.

Purchase SFT



Purchase CFolioItem (WETH/WOWS UniV2 LP token stake)

```
WOWSSFTMinter::mintCFolioItemSFT(...BaseCardSFT)
  WOWSERC1155::mint (CFolioItemSFT)
  CFolioItemHandlerLP::setupCFolio
    CFolioItemHandlerLP::deposit
      CFolioFarm::addAssets
  WOWSERC1155::transferFrom(WOWSSftMinter, TradeFloor, CFolioItemSFT)
    TradeFloor::onERC1155Received (mint TradeFloor NFT)
  TradeFloor::TransferFrom(WOWSSftMinter, BaseCardSFT, CFolioItemSFT)
    WOWSCryptoFolio::onERC1155Received
    CFolioItemHandler::onCFolioItemsTransferFrom
      SFTEvaluator::rewardRate
    CFolioFarm::addShares
```

Purchase CFolioItem (SC deposit)

```
WOWSSFTMinter::mintCFolioItemSFT(...BaseCardSFT)
  WOWSERC1155::mint (CFolioItemSFT)
  CFolioItemHandlerSC::setupCFolio
    CFolioItemHandlerSC::deposit
      CFolioFarm::addAssets
  WOWSERC1155::transferFrom(WOWSSftMinter, TradeFloor, CFolioItemSFT)
    TradeFloor::onERC1155Received (mint TradeFloor NFT)
  TradeFloor::TransferFrom(WOWSSftMinter, BaseCardSFT, CFolioItemSFT)
    WOWSCryptoFolio::onERC1155Received
    CFolioItemHandler::onCFolioItemsTransferFrom
      SFTEvaluator::rewardRate
    CFolioFarm::addShares
```

Trade base SFT on NFT marketplace

```
WOWSERC1155::transferFrom(WOWSSftMinter, TradeFloor, CFolioItemSFT)
  TradeFloor::onERC1155Received (mint TradeFloor NFT)
```

Trade CFolioItem on NFT marketplace

```
TradeFloor::TransferFrom(BaseCardSFT, WALLET, CFolioItemSFT)
  CFolioItemHandler::onCFolioItemsTransferFrom
    SFTEvaluator::rewardRate
  CFolioFarm::removeShares
```